

ESP32-DIV — Complete Hardware Guide

Firmware: ESP32-DIV · **Upstream:** [cifertech/ESP32-DIV](#) (MIT) **Chips:** ESP32-S3 (DIV v2, current), ESP32 (DIV v1, legacy) · **Cyber Controller profile:** [esp32-div](#) (esptool backend, app image @ 0x10000, per-chip flash_freq: S3 80m / classic 40m, suicide-capable) **This guide:** purchase → build/assemble → flash → integrate into Cyber Controller → use → troubleshoot.

1. Overview

ESP32-DIV is CiferTech's handheld "Swiss-army-knife" wireless multi-tool — a single ESP32 device with an on-screen menu UI that bundles **WiFi** (scanner, packet monitor with waterfall + PCAP, deauther, deauth detector, beacon spam, probe flood, captive portal), **BLE** (scanner, sniffer, spoofer, "Sour Apple", jammer, Rubber-Ducky HID), a **2.4 GHz spectrum analyzer/scanner and jammer driven by NRF24 modules**, and **Sub-GHz capture/replay/jam via a CC1101 module**. Newer builds also expose IR, RFID/NFC (PN532), and GPS wardriving (verify per firmware version — §9). The radios beyond the ESP32's own WiFi/BLE live on a **stackable RF shield** (3× NRF24 + 1× CC1101). Cyber Controller flashes the firmware and connects to it as a serial device for control and logging.

2. Legal & Safety

Authorized testing only. ESP32-DIV's transmit features operate in **license-free ISM bands but are not license-free to abuse**: WiFi deauth, beacon/probe spam, captive portal, BLE spam/jam ("Sour Apple", BLE Jammer), the **NRF24 "Proto Kill" 2.4 GHz jammer**, and the **CC1101 Sub-GHz jammer/replay** are illegal against people/devices/networks you do not own or have **written permission** to test in most jurisdictions (US: CFAA + FCC Part 15 prohibit willful interference/jamming; EU and others similar). Treat the spectrum analyzers and passive scanners/sniffers as the default-safe modes; treat every jam/replay/deauth/spam mode as **analysis-or-authorized-use-only**. Cyber Controller flags dangerous verbs for this profile and (unless you disable warnings) confirms before sending. Passive RF capture may still be regulated where you live — check local law.

3. Hardware & Purchasing

ESP32-DIV is an open-hardware design (PCB + RF shield). You can **buy a pre-assembled unit** or **build from the published gerbers/BOM**. Two firmware/hardware generations exist:

Generation	MCU	Notes	Cyber Controller board
DIV v2 (current)	ESP32-S3 (8MB per profile; commercial boards report 16MB — verify your unit)	2.8" ILI9341 + XPT2046 touch, IP5306 battery mgmt, CP2102 USB-serial, pogo-pin stacked RF shield, 4× WS2812, SMA antenna	ESP32-S3 (DIV v2, current) — dio / 80m
DIV v1 (legacy)	ESP32 (classic WROOM, 4MB per profile)	Original compact design, push-button nav; earlier core/partition layout	ESP32 Generic (DIV v1, legacy) — dio / 40m

Pre-built (easiest): search "**ESP32-DIV v2**" at hardware resellers — e.g. M5Shark lists an *ESP32-DIV V2.0 Development Board* (ESP32-S3, RF shield with 3× NRF24L01 + 1× CC1101, 2.8" ILI9341 touch, SD slot, WS2812 + buzzer, USB-C, SMA antenna, enclosure; **battery and SD usually not included**). Confirm stock/price at purchase time — these sell out (verify: vendor listing in §9).

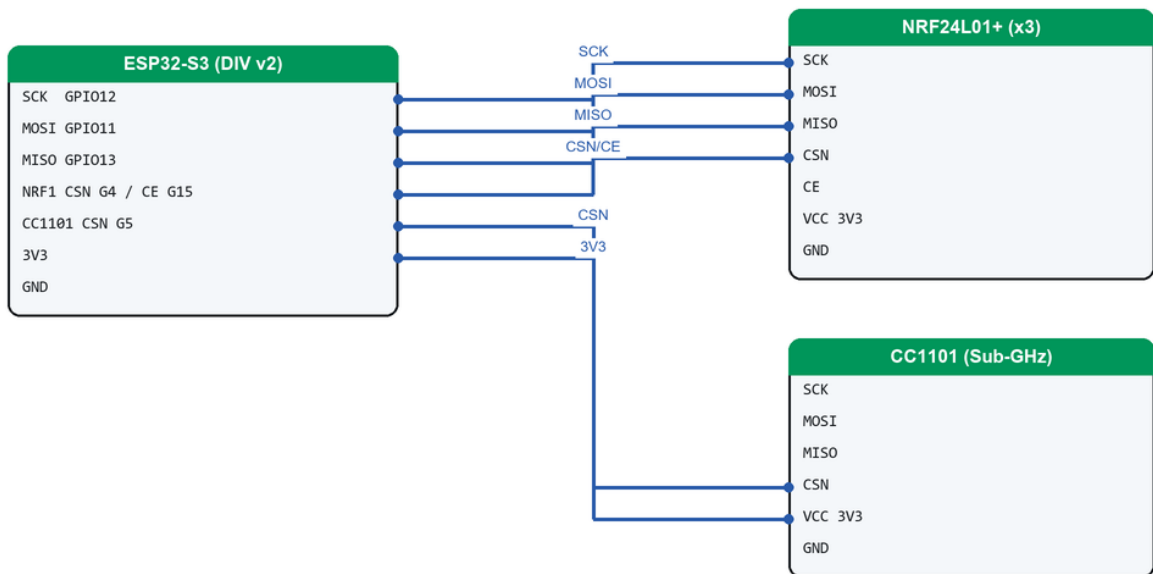
Build-your-own (PCB + modules): the project repo/wiki publishes the schematic, board files, and BOM — order the main board + RF shield from a fab (e.g. JLCPCB) using the repo's gerbers, then source the modules:

Part	What to search	Notes
2.4 GHz radios	"NRF24L01" (or "NRF24L01+ PA LNA" for the SMA/long-range variant) ×3	DIV uses three NRF24 modules for wideband 2.4 GHz scan/jam
Sub-GHz radio	"CC1101 433MHz module" (also 315/868/915 variants by region)	one CC1101 for SubGHz capture/replay/jam
Display	"ILI9341 2.8 inch SPI TFT XPT2046 touch"	2.8" 240×320, resistive touch
Optional	"PN532 NFC module", "Neo-6M GPS", microSD (FAT32)	for RFID/NFC + wardriving builds (version-dependent)

Accessories: a **data-capable USB cable** (USB-C on v2), a **FAT32 microSD** for PCAP/profile/ducky logging, region-correct **SMA antennas** for the NRF24 PA/LNA and CC1101, and a **compatible Li-ion battery** (sourced separately — use a certified cell; the IP5306 handles charge/boost). Buy from reputable sellers (AliExpress/Amazon/vendor stores using the search strings above) — **do not** trust invented URLs; confirm current listings (§9).

4. Building / Assembly

Wiring — ESP32-DIV: NRF24 + CC1101 on the shared SPI bus (v2 GPIO)



Radios share SCK/MOSI/MISO; each has its own CS. NRF24 #2/#3 use separate CSN/CE GPIO (see the connection table). Power radios from a clean 3.3V; add a 10uF cap per NRF24 VCC/GND to avoid brownouts. Verify exact GPIO against the upstream v2 schematic.

Pin layout & connections — verify exact GPIO against the upstream schematic and the table below.

- **Pre-built v2:** no assembly — connect a USB-C data cable and (optionally) install a battery + microSD.
- **DIY build (main board + RF shield):** solder/populate per the repo BOM, then mate the **RF shield to the main board via the pogo-pin headers** (v2's spring-loaded stack — no bulky connectors). Seat the **3× NRF24** and **1× CC1101** on the shield; attach SMA antennas; insert the ILI9341/XPT2046 display.
- **Wiring (ESP32-S3 / v2 — verify against the current Schematics wiki, §9):** the RF radios share one SPI bus; the display/touch use a second SPI bus. Exact GPIO from the v2 schematic:

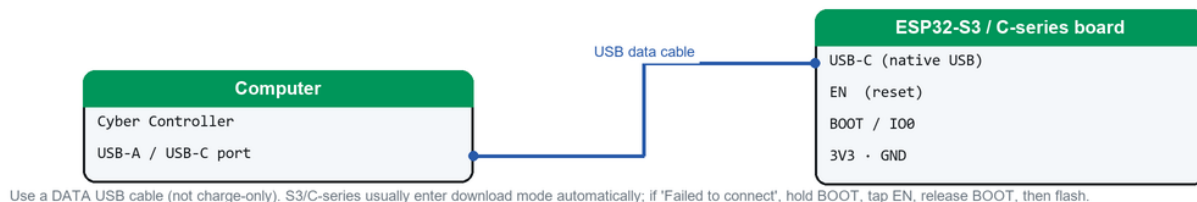
Function	Pin(s)
Radio SPI bus (shared)	SCK 12, MOSI 11, MISO 13
NRF24 #1	CSN 4, CE 15
NRF24 #2	CSN 48, CE 47
NRF24 #3	CSN 21, CE 14
CC1101	CS 5, GDO0 6, GDO2 3
microSD	CS 10 (shares radio SPI), card-detect 38
ILI9341 TFT	SCK 36, MOSI 35, MISO 37, CS 17, DC 16, BL 7

XPT2046 touch	T_CS 18 (shares display SPI)
Misc	Buzzer 2 , WS2812 NeoPixel 1 , buttons via PCF8574 I ² C expander

(DIV v1/classic-ESP32 uses a different, button-driven pinout — verify against the v1 schematic; do not reuse the S3 pins above on v1.) - **Drivers:** install the **CP2102** USB-serial driver (some clones use **CH340**). Without it no COM/tty port appears. (S3 native-USB units may enumerate without a driver — verify your board.) - **Touch calibration:** first boot (touch builds) runs a four-corner XPT2046 calibration; recalibrate any time from **Tools** → **Touch Calibration**.

5. Flashing & First Run (via Cyber Controller)

Flashing connection — ESP32 (native USB)



How to connect the board to flash it via Cyber Controller (native-USB ESP32).

1. Connect the device by USB; open Cyber Controller → **Flash** tab.
2. **Port:** pick the device's COM/tty (click *Refresh* if missing → driver issue, §8).
3. **Firmware Profile:** ESP32-DIV.
4. **Board / chip:** choose **ESP32-S3 (DIV v2, current)** for the S3 hardware, or **ESP32 Generic (DIV v1, legacy)** for the original classic-ESP32 unit. This sets the per-chip flash_freq (**S3 80m / classic 40m**), both **dio**. Picking the wrong generation will fail or boot-loop.
5. Click **Flash**. Cyber Controller (esptool backend) fetches the latest GitHub release **app image** and writes it at **0x10000**, pulling the boot chain from the repo tree — **bootloader** (standard offset for the chip), **partitions @ 0x8000**, and **boot_app0 @ 0xe000**. First boot shows the ESP32-DIV menu (two columns × four rows of tool hubs) on the TFT.

Alternate flashers (reference): CiferTech's browser flasher ([cifertech.github.io/ESP32-DIV / FirmwareHub](https://cifertech.github.io/ESP32-DIV/)) uses WebSerial + esptool.js, auto-detects the chip ID, and picks the v1/v2 build; Espressif Flash Download Tool works too (app .bin @ 0x10000, partitions @ 0x8000, DIO, 40 MHz for v1); and the device self-updates via **Tools** → **Firmware Update** → **OTA / SD** (rename to `firmware.bin` on a FAT32 card). Use Cyber Controller's Flash tab for the integrated path.

6. Integrate into Cyber Controller

- **Profile:** `esp32-div` (esptool backend; image model = **multi-file-offsets**; app @ `0x10000`, partitions @ `0x8000`, boot_app0 @ `0xe000`, bootloader at the chip's standard offset). Release assets are resolved from GitHub (`github_release`, `.bin` asset). **Suicide/erase-capable** profile.
- **Control:** open the **Devices** tab → select the port → set **Firmware = ESP32-DIV** (Pro mode; Simple mode auto-selects it) → **Connect** at **115200** baud. The `esp32-div` protocol parser reads the device's serial output and the persistent terminal colorizes it per device.
- **Cross-device:** scan results can feed the shared Targets pool / `target.added` events for auto-routing rules to other connected devices; **Dead Man's Switch** is supported for this profile.
- **Backup first:** use **Backup** in the Flash tab to dump current flash before overwriting (and **Erase Flash** if you need a clean slate before switching generations).

7. Usage (end-to-end)

ESP32-DIV is primarily a **standalone, on-device** tool (navigate the TFT menu / touch / buttons); Cyber Controller manages flashing, serial logging, and cross-device coordination. 1. **WiFi recon**: open **WiFi** → **Scanner** (APs with channel/RSSI/security) or **Packet Monitor** (waterfall + PCAP to SD). **Deauth Detector** passively flags deauth activity. 2. **BLE recon**: **Bluetooth** → **BLE Scanner / Sniffer** for nearby advertisements (RSSI + heuristics). 3. **2.4 GHz analysis**: **2.4 GHz (NRF24)** → **Scanner** shows per-channel energy across the band — a true spectrum view using the three NRF24 radios. 4. **Sub-GHz**: **SubGHz (CC1101)** → **Replay** learns/replays a simple remote (authorized only); **Saved Profiles** replays signals stored on SD. 5. **Authorized active tests (lab/written-permission only)**: WiFi Deauther / Beacon Spammer / Probe Flood / Captive Portal; BLE Spoofer / Sour Apple / Jammer; NRF24 **Proto Kill**; CC1101 **Jammer**; BLE **Rubber Ducky** (payloads from /duffy on SD). See §2. 6. **Logging**: capture/PCAP/profiles/wardrive data write to the microSD; pull the card or use the device's SD tools to retrieve. 7. **Stop / disconnect**: exit the active mode on-device; in Cyber Controller **Disconnect** to free the port.

8. Troubleshooting

- **No COM/tty port:** install **CP2102** (or **CH340**) driver; use a **data** cable; on Linux add your user to `dialout` and replug. S3 native-USB units: try the other USB connector if the board has two.
- **Flash fails / "Failed to connect":** enter download mode — **hold BOOT, tap RESET, release** — then flash; lower the flash baud in Settings; close any serial monitor (incl. the Devices terminal) holding the port; for v1 binary flashing drop SPI speed to 20 MHz.
- **Wrong generation:** flashing the S3 image to a classic ESP32 (or vice-versa) fails or boot-loops — pick the matching **board/chip** in §5; **Erase Flash** before switching.
- **Blank screen / wrong colors after flash:** confirm v2 (ILI9341/XPT2046) vs v1 hardware; recheck display wiring (§4); for touch misalignment run **Tools** → **Touch Calibration**.
- **NRF24 modules not detected / weak 2.4 GHz scan:** reseat all three NRF24 on the shield, verify the shared SPI bus + each module's CE/CSN, ensure stable 3.3 V (NRF24 PA/LNA needs adequate current — add the decoupling cap if your modules brown out), and check antenna fit.
- **CC1101 silent / no SubGHz:** confirm CS/GDO0 wiring, the correct **frequency-band variant** (433 vs 315/868/915) for your region, and antenna; verify SubGHz tx is lawful where you are (§2).
- **SD features fail:** card must be **FAT32**; reseat; for OTA/SD update name the file exactly `firmware.bin` in the card root.
- **Boot-loops / brick:** **Erase Flash** then re-flash; verify the board's real flash size matches the selected generation.

9. Sources

- Upstream repo: <https://github.com/cifertech/ESP32-DIV> (README, releases, MIT license).
- Wiki: **Hardware**, **Features**, **Schematics** (pin/wiring), **Firmware-Upload** (flash methods) — <https://github.com/cifertech/ESP32-DIV/wiki>. Pin table above is from the v2 ESP32-S3 schematic; **verify** against the current Schematics page (pins can change between revisions).
- Author write-ups: CiferTech blog ("*ESP32-DIV: Your Swiss Army Knife...*", "*ESP32-DIV v2...*") at <https://cifertech.net>; Elektor / Hackster coverage of ESP32-DIV v2.
- Browser flasher: <https://cifertech.github.io/ESP32-DIV> and FirmwareHub <https://cifertech.github.io/FirmwareHub> (WebSerial + esptool.js).
- Purchasing (pre-built v2): vendor listing e.g. M5Shark *ESP32-DIV V2.0* — **verify** stock/price/inclusions at purchase time. Modules: search "**NRF24L01**" / "**NRF24L01+ PA LNA**", "**CC1101 433MHz module**", "**ILI9341 2.8 XPT2046**" from reputable sellers (no fixed URL — listings change).
- Cyber Controller profile: `src/config/profiles/esp32_div.json` (id `esp32_div`, protocol `esp32-div`, esptool, offsets, per-chip flash_freq, boot-file tree).
- **Verify** flash size (8 vs 16MB on S3 units), v1 exact MCU/flash, and the extended IR/RFID/GPS feature set against your specific board revision and firmware release before relying on them.